Semana 6 passo a passo da integração FIU e Hash

Projeto CI Digital Inatel, T25S2, Grupo 2. Samuel: Hash Unit. Bruno: Flow Identification Unit. Herman: Packet Parser. Semana 6: 19 a 25 de outubro de 2026. Revisão: 22/10/2026.

O arquivo `Semana_6.docx` é a referência principal desta entrega. A seção 2.3 pede implementar recepção e envio nos dois módulos e simular sua integração com metadados conhecidos. Este pacote entrega os dois RTLs atualizados, a ligação direta entre eles e testes executados com comparação automática dos resultados. A documentação explica o processo para que Samuel possa acompanhar as decisões e compartilhá-las com Bruno.

1. Transforme as orientações da Semana 6 em verificações concretas.

A seção 3 do documento determina clock e reset síncrono, validade sem acrescentar `ready` e uma regra de recepção: “somente aceitar uma nova leitura do barramento após valid ir para nível lógico baixo”. Essa regra tem prioridade sobre o comportamento dos pacotes anteriores.

Entregável da seção 2.3	Material desta entrega	Situação
Recepção e envio nos dois módulos	`rtl/flow_identification_unit.sv` e `rtl/hash_unit.sv`	Implementados e simulados
Simulação FIU e Hash integrados	`rtl/fiu_hash_top.sv`, testbench e log da integração	Passou
Casos com esperado e obtido	`validation/casos_integracao.csv` e CSVs brutos	181 casos da coleção conferidos
Falhas e correções	`falhas_e_correcoes_semana6.md` e logs de regressão	Dois comportamentos antigos reproduzidos e corrigidos

O diagrama de interfaces indicado na Semana 6 foi conferido. Ele apresenta IPs de 32 bits, portas de 16, protocolo de 8, comprimento de 16, timestamp de 80, chave de 104 e índice de 16. Os documentos e códigos anteriores servem como base de migração; o protocolo atual é o da Semana 6.

O teste representa a origem dos metadados e o consumidor final por estímulos e monitores. Entre eles, a FIU e o Hash são módulos RTL completos e conectados diretamente. Assim, o escopo corresponde à seção 2.3.

2. Entenda o que passa em cada interface.

A FIU recebe os sete campos abaixo. Clock, reset e validade acompanham a interface como controle.

Campo	Bits	Porta de entrada da FIU
IP de origem	32	`src_ip_in`
IP de destino	32	`dst_ip_in`
Porta de origem	16	`src_port_in`
Porta de destino	16	`dst_port_in`
Protocolo	8	`protocol_in`

Campo	Bits	Porta de entrada da FIU
Comprimento	16	`packet_length_in`
Timestamp	80	`timestamp_in`

São 200 bits de dados. A FIU reúne os primeiros cinco campos em uma chave de 104 bits. Comprimento e timestamp continuam associados ao pacote e são preservados.

```
flow_key = {src_ip, dst_ip, src_port, dst_port, protocol};
```

Campo	Posição na chave
IP de origem	`[103:72]`
IP de destino	`[71:40]`
Porta de origem	`[39:24]`
Porta de destino	`[23:8]`
Protocolo	`[7:0]`

A FIU mantém a direção do fluxo: origem e destino não são ordenados nem trocados. A mesma 5-tuple produz a mesma chave, mesmo quando tamanho e timestamp variam. Cada pacote continua sendo uma transferência distinta.

O Hash recebe os 200 bits, acrescenta o índice de 16 bits e publica 216 bits de dados. Somente os 104 bits da chave entram no cálculo do CRC. O timestamp é preservado integralmente; não é convertido nem usado no hash.

3. Aplique a regra de uma captura por período alto de valid.

Um `valid` alto indica uma transferência até que o receptor observe um nível baixo em uma borda de subida. Manter o sinal alto por vários clocks não cria novos pacotes. Nesta implementação, os emissores publicam suas saídas por um ciclo; os receptores também suportam um nível alto prolongado sem repetir a leitura.

Situação observada na borda	Ação do receptor
`valid = 1`, período alto ainda não capturado e módulo disponível	Captura todos os dados e registra `start = 1`
`valid = 1`, mesmo período alto já observado	Ignora a repetição; não captura novamente
`valid = 0`	Rearma a recepção para o próximo período alto
Novo período alto enquanto ocupado	Violação da capacidade da implementação; diagnosticada na simulação
Reset ativo	Cancela a operação e reinicializa o controle

O registrador interno `valid_seen` guarda se o período alto atual já foi observado. Ele fica em um até aparecer uma borda com `in_valid = 0`. A condição principal da captura é:

```
if (in_valid && !valid_seen && capture_allowed) begin
    // Capturar os campos do mesmo pacote.
    start <= 1'b1;
end
```

`capture_allowed` indica que o módulo está livre e não há `start` pendente. É um sinal interno, sem criar uma porta de `ready`. O registro `valid_seen` é atualizado mesmo se uma transferência nova chegar em

momento indevido; isso evita aceitar essa mesma validade mais tarde como se fosse uma entrada nova.

Como tudo é síncrono, o nível baixo precisa ser observado em uma borda. A recepção não usa `posedge valid` como clock e não depende de uma transição assíncrona do sinal.

4. Acompanhe a recepção e o envio implementados na FIU.

Abra `rtl/flow_identification_unit.sv`. A recepção guarda os sete campos juntos e ativa `start` por um ciclo. A máquina de estados observa esse `start` na borda seguinte, monta a chave com os campos já capturados e publica o registro.

Borda relativa à captura FIU	Ação	Situação após a borda
`E0`	Captura os sete campos	`start = 1`; estado ainda `IDLE`
`E1`	FSM consome `start` e registra a saída	`out_valid = 1`; estado `PUBLISH`
`E2`	Hash lê a saída da FIU	A FIU encerra a publicação e volta a `IDLE`
`E3`	Primeira nova captura local possível	Exige período de validade rearmado

As atribuições <= são não bloqueantes. Por isso, a FSM lê o `start` registrado na borda anterior. Essa separação garante que o processamento use um conjunto de dados já preservado, e não uma mistura de campos externos que possam mudar depois.

O intervalo local de três clocks descreve a FIU sozinha. Ao conectá-la ao Hash, o caminho deve respeitar a capacidade do Hash, que exige intervalo maior. O teste integrado verifica esse caso.

5. Faça a ligação direta entre FIU e Hash.

Abra `rtl/fiu_hash_top.sv`. Esse arquivo instancia `u_fiu` e `u_hash` e usa quatro ligações para os dados e a validade. Os dois módulos compartilham clock e reset.

Saída da FIU	Entrada do Hash	Bits
`flow_key_out`	`flow_key_in`	104
`packet_length_out`	`packet_length_in`	16
`timestamp_out`	`timestamp_in`	80
`out_valid`	`in_valid`	1

A borda `E2` é simultaneamente a leitura da saída da FIU e a captura da entrada do Hash. Não existe uma cópia manual intermediária feita pela testbench. O monitor compara os 200 bits nessa ligação e confere, após a atualização da borda, os registradores efetivamente capturados pelo Hash.

As saídas da FIU podem permanecer com valores antigos quando sua validade é zero. O Hash usa a validade para determinar a captura. A presença de um valor no barramento, por si só, não representa um pacote.

6. Acompanhe a recepção, o CRC e a publicação no Hash.

O Hash conserva a interface de dados do pacote anexo e recebe o registro uma única vez por período alto de validade. A atualização da Semana 6 acrescenta `valid_seen`; a função de CRC permanece com os mesmos parâmetros e vetores de referência.

Parâmetro do CRC	Valor
Perfil	CRC-16/CCITT-FALSE, também conhecido como CRC-16/IBM-3740
Polinômio	`0x1021`
Inicialização por chave	`0xFFFF`
Reflexão de entrada e de saída	Não
XOR final	`0x0000`
Dados processados	13 bytes da chave, MSB primeiro
Índice de saída	Todos os 16 bits do CRC

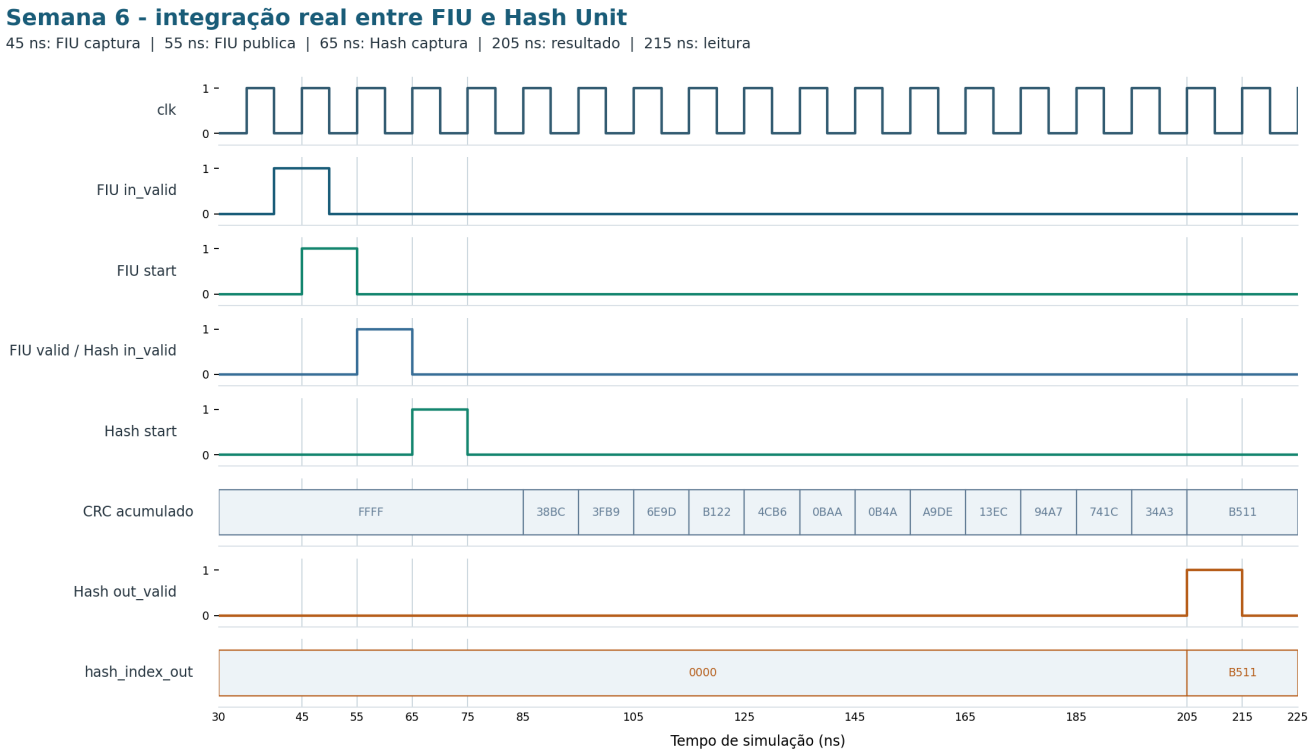
Após a captura, o `start` registrado inicia a FSM. `CALC` processa um byte por clock; a função contém oito passos combinacionais para os bits desse byte. Depois do byte 12, `result_reg <= next_crc` inclui a contribuição do último byte. O estado `RESULT` publica `out_valid` por um ciclo.

O exemplo TCP usa origem `192.168.1.10:50000`, destino `192.168.1.20:443` e protocolo 6. A chave esperada é `C0A8010AC0A80114C35001BB06`; o CRC esperado e o obtido são `B511`. Na primeira transação integrada, comprimento 60 e timestamp `8000000000000000007B` também foram preservados.

7. Leia a temporização da integração completa.

Considere `E0` como a captura inicial dos metadados na FIU. O resultado de uma borda passa a estar disponível depois da atualização dos registradores; o receptor seguinte o amostra na borda posterior.

Borda na integração	Evento	Primeiro exemplo
`E0`	FIU captura os metadados e gera `start`	45 ns
`E1`	FIU monta a chave e ativa sua validade	55 ns
`E2`	Hash captura a saída da FIU e gera seu `start`	65 ns
`E3`	FSM do Hash consome `start` e inicia o cálculo	75 ns
`E4` a `E15`	Hash processa bytes 0 a 11	85 a 195 ns
`E16`	Hash processa byte 12 e disponibiliza o resultado	205 ns
`E17`	Consumidor lê o registro de saída completo	215 ns



Fonte: transições do VCD desta execução. Clock = 10 ns. Chave A = C0A8010AC0A80114C35001BB06.

Primeira transação da integração extraída do VCD.

A latência de ponta a ponta até a leitura é 17 clocks: dois da FIU e 15 do Hash. A disponibilidade ocorre após 16 clocks. Esses valores são diferentes do intervalo entre novas entradas.

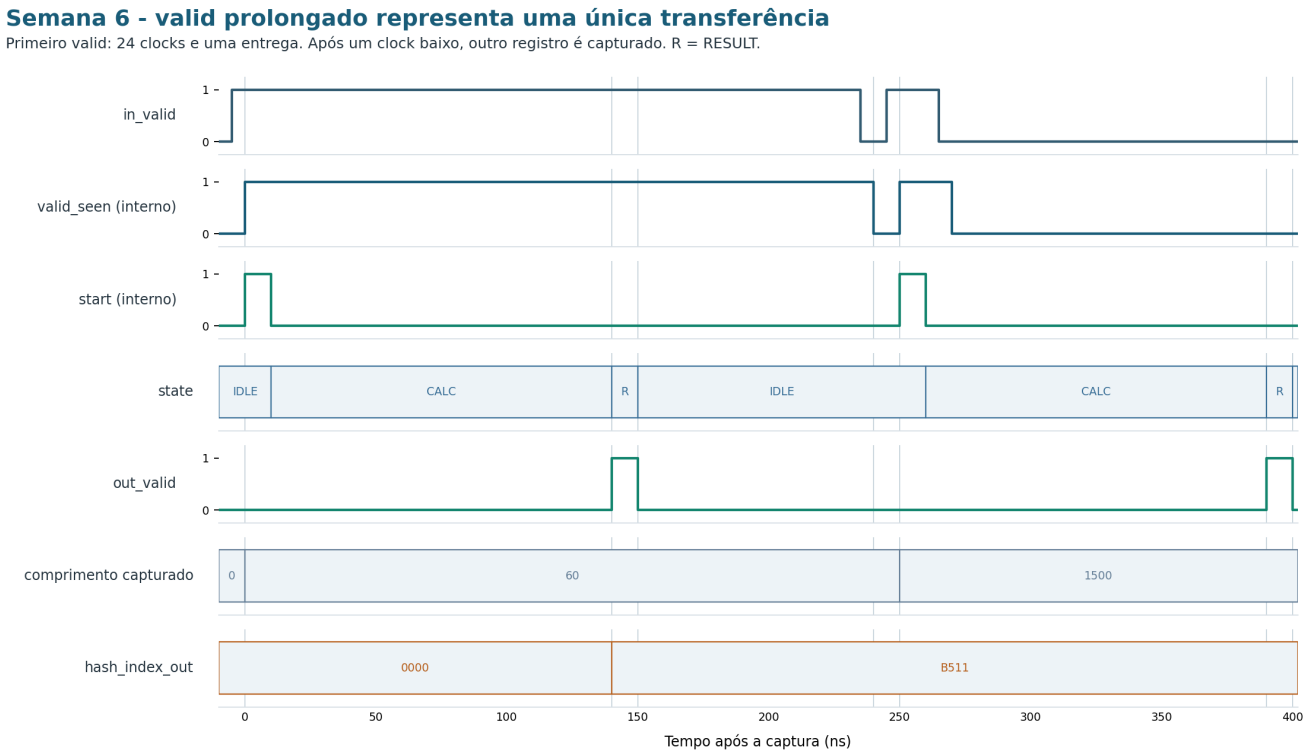
O caminho integrado aceita entradas na FIU a cada 16 clocks. Enquanto o Hash termina um registro, a FIU pode preparar o próximo. O teste exerceu essa sobreposição com dois registros em andamento, um em cada módulo. Isso não acrescenta FIFO ao hardware.

Regra para o produtor: se cada entrada usa um pulso de um ciclo, separe as capturas na FIU por pelo menos 16 clocks. Se `valid` durar vários ciclos, também é necessário pelo menos um clock com validade baixa antes da transferência seguinte. Um período alto de 24 clocks exige intervalo mínimo de 25 clocks para permitir esse rearme.

A folga entre pacotes informada pelo professor sustenta a implementação sem FIFO. O valor de 16 clocks quantifica a condição desta versão de integração. Com o clock de teste de 10 ns, o intervalo é 160 ns e a latência até a leitura é 170 ns. A análise física da FPGA pertence à etapa de síntese e temporização.

8. Observe a captura única mesmo com valid prolongado.

A testbench mantém `valid` alto por dois, cinco e 24 clocks. Em cada período alto, espera uma única saída. O caso de 24 clocks é importante porque o Hash termina o processamento antes de a validade voltar a zero: mesmo livre, ele não deve capturar outra vez o mesmo registro.



Fonte: transições do VCD desta execução. Clock = 10 ns. Chave A = C0A8010AC0A80114C35001BB06.

Valid alto prolongado e rearme após nível baixo, a partir do VCD do Hash.

Na figura, o primeiro pacote é capturado em tempo relativo zero. O resultado é publicado aos 140 ns e lido aos 150 ns. `valid_seen` continua ativo enquanto a entrada permanece alta. Depois de uma borda baixa, outro período alto captura o segundo registro, com seu próprio comprimento. Ambos os receptores passaram nesse comportamento em seus testes unitários.

Dados de entrada também são alterados quando a validade é zero. Os registros capturados preservam os campos corretos até o resultado, comprovando a associação entre cada chave, comprimento, timestamp e índice.

9. Entenda os testes executados e os valores esperados.

O modelo `python/reference_hash.py` e os arquivos `hash_vectors.mem` e `hash_vectors.csv` são idênticos aos do ZIP anexo. Os 181 vetores contêm 13 casos dirigidos, 104 chaves de bit único e 64 casos pseudoaleatórios com semente 392.

O esperado é produzido por `binascii.crc_hqx(key, 0xFFFF)`, da biblioteca padrão Python, antes da simulação. Os valores obtidos são escritos diretamente pela testbench a partir das saídas do RTL. O executor compara novamente os CSVs ao final.

Execução	Registros entregues e conferidos	Latência até a leitura
FIU isolada	186	2 clocks
Hash isolado	186	15 clocks
FIU e Hash conectados	186	17 clocks

Em cada execução, são 181 registros da coleção, um teste inicial e quatro operações com validade prolongada. Portanto, há 558 entregas conferidas no total. Dessas, 372 incluem comparação do CRC nas execuções do Hash isolado e da integração. A FIU isolada compara os 200 bits de dados, conforme sua função.

Os padrões artificiais de todos os bits, bit único e protocolo arbitrário exercitam as larguras dos barramentos. São metadados simulados; a filtragem dos pacotes do sistema cabe ao Parser. Os timestamps são preservados como 80 bits, inclusive os padrões extremos de teste.

Os casos de mesma chave com metadados diferentes e de chaves diferentes com o mesmo índice `B511` passaram. A Hash Unit conserva a chave para os módulos seguintes; o tratamento de colisão pertence à frente do Controller.

10. Confira os resets e as falhas registradas.

O reset continua síncrono e ativo baixo, com nome `reset_n`, preservando as duas interfaces de origem. O diagrama usa o rótulo genérico `rst`; esta implementação explicita a polaridade em seu contrato de integração. Os módulos conectados devem aplicar essa mesma convenção ou fazer a adaptação correspondente na ligação do sistema.

O teste integrado capturou 190 entradas: entregou 186 e cancelou quatro por reset. Um reset cancela o registro recém-capturado na FIU; outro interrompe o processamento no Hash; o último cancela dois registros, um com resultado disponível no Hash e outro recém-capturado na FIU.

A conta fecha: `190 = 186 entregues + 4 cancelados + 0 pendentes`. O monitor registrou 188 transferências da FIU e 188 capturas pelo Hash. Duas entradas foram canceladas antes dessa ligação; duas que já haviam chegado ao Hash foram canceladas antes da leitura final.

O relatório de falhas distingue incompatibilidades antigas de testes negativos planejados. O Hash anterior abortava com validade prolongada; a FIU anterior podia repetir o mesmo registro sem nível baixo. Ambos os comportamentos foram reproduzidos com os códigos históricos e corrigidos na versão atual. O mesmo estímulo passa nos RTLs da Semana 6.

Os testes negativos atuais apresentam uma transferência nova durante ocupação. Cada módulo a detecta, e a integração confirma que publicar na FIU a cada três clocks pode atingir o Hash ainda ocupado. Esses testes devem terminar com `PROTOCOL_VIOLATION`; o executor verifica que a falha foi a esperada.

11. Reproduza os testes e encontre os arquivos da entrega.

Extraia o ZIP e abra um terminal na pasta `semana6_fiu_hash`. Com Python 3.10 ou superior, Verilator, compilador C++ e `make`, execute:

```
python3 python/run_tests.py
python3 python/run_regressions.py
python3 python/report_results.py
```

Nos dois primeiros comandos, use `--tool /caminho/para/verilator` se o executável estiver fora do `PATH`. O primeiro executa as três simulações nominais e os três testes negativos de capacidade. O segundo reproduz as duas incompatibilidades históricas e demonstra suas correções. O terceiro organiza os 181 casos da integração com os campos de entrada e os resultados observados.

As figuras podem ser refeitas, com `matplotlib` disponível, usando `python3 python/render_waveforms.py`. Os VCDs e os testes funcionais não dependem dessa biblioteca de desenho.

Bruno pode repetir o teste nominal da integração no ModelSim ou Questa a partir da pasta do pacote:

```
do run_modelsim.do
```

Para os módulos isolados, execute `do run_modelsim.do fiu` ou `do run_modelsim.do hash`, encerrando a simulação anterior antes de trocar o teste. O script grava os registros em `validation/modelsim/`. A

execução comprovada nesta entrega foi realizada no Verilator; o script ModelSim foi preparado para reprodução e não foi executado aqui.

Arquivo ou pasta	Uso
`rtl/flow_identification_unit.sv`	Recepção, concatenação e envio da FIU
`rtl/hash_unit.sv`	Recepção única, CRC e publicação do Hash
`rtl/fiu_hash_top.sv`	Ligação direta entre os dois módulos
`tb/metadata_testbench.sv`	Estímulos, scoreboard e monitores compartilhados
`tb/tb_fiu_hash_integration.sv`	Seleção do teste integrado
`tb/tb_flow_identification_unit.sv`	Seleção do teste unitário da FIU
`tb/tb_hash_unit.sv`	Seleção do teste unitário do Hash
`validation/casos_integracao.csv`	181 casos com entradas, esperado e obtido
`validation/integracao_obtidos.csv`	186 entregas observadas, incluindo testes extras
`validation/integracao_transferencias.csv`	188 transferências reais FIU para Hash
`validation/summary.json`	Contadores, versão e hashes dos arquivos simulados
`regressions/`	Códigos históricos e testes de reprodução das incompatibilidades

Os códigos em `regressions/baseline/` são arquivos históricos. Eles não participam do top integrado. Compile o conjunto de `rtl/` para a versão da Semana 6; o executor mantém as bibliotecas e os binários de regressão separados.

12. Apresente as decisões e o alcance ao grupo.

A entrega para alinhar Bruno, Samuel e a equipe seguinte contém: recepção única por período alto de validade; rearme após baixo amostrado; captura conjunta de 200 bits; `start` interno registrado; publicação de um ciclo; reset síncrono ativo baixo; e intervalo integrado mínimo de 16 clocks para pulsos de um ciclo.

Uma explicação para a reunião é: “Na Semana 6 conectamos a FIU ao Hash no RTL. A FIU recebe os metadados, forma a chave e publica para o Hash. Os dois receptores capturam uma vez por período alto de valid e só rearmam depois de observar baixo. A integração passou nos 181 vetores originais, preservando todos os dados. Também testamos validade prolongada, resets e chegadas antes do tempo permitido.”

As três simulações nominais terminaram sem divergências. Os arquivos identificam exatamente a versão que foi testada. A validação funcional usa metadados e consumidor simulados; a conexão com o Parser e o Controller completos e a verificação física da FPGA serão etapas próprias da integração do sistema.

As alterações de protocolo estão documentadas neste guia e nos diffs para alinhamento com as equipes, conforme a seção 3 da Semana 6. A entrega não pressupõe uma aprovação coletiva já realizada. O código da FIU foi migrado a partir do ZIP público de Bruno na revisão consultada; possíveis versões locais posteriores devem ser comparadas com esta implementação.

Referências verificadas: `Semana_6.docx`, seções 2.3 e 3, fornecido por Samuel; ZIP anexo da Hash Unit com valid/captura/start; diagrama `Interfaces.pdf` no endereço indicado pela Semana 6; ZIP público `semana4_fiu_BRUNO.zip` e estado do repositório no commit `ecec6bd9d1af1969f7122d55d71a7422bf2cbcbe`. Os exemplos da pasta Drive não foram verificados nesta entrega.